PRACTICAL - 06

Part A — Practice Programs (Hour 1)

| | |
|---|---|
| PROBLEM1 | 1. Form validator: define @NotBlank and @MaxLength(int) for fields and annotate a SignupForm. Write a checker that uses reflection to read each field's annotations and returns the list of validation errors for a given object. |
| CODE | import java.lang.annotation.*;<br>import java.lang.reflect.Field;<br>import java.util.*;<br><br>@Retention(RetentionPolicy.RUNTIME)<br>@Target(ElementType.FIELD)<br>@interface NotBlank {<br>}<br><br>@Retention(RetentionPolicy.RUNTIME)<br>@Target(ElementType.FIELD)<br>@interface MaxLength {<br>  int value();<br>}<br><br>class SignupForm {<br><br>  @NotBlank<br>  @MaxLength(10)<br>  String username;<br><br>  @NotBlank<br>  @MaxLength(20)<br>  String email;<br><br>  SignupForm(String username, String email) {<br>  this.username = username;<br>  this.email = email;<br>  }<br>}<br><br>public class Main {<br><br>  public static void main(String[] args) throws Exception {<br><br>  SignupForm form = new SignupForm("", "abcdefghijklmnop@gmail.com");<br><br>  List<String> errors = new ArrayList<>(); |

```
        for (Field field : form.getClass().getDeclaredFields()) {

            field.setAccessible(true);

            String value = (String) field.get(form);

            // Check NotBlank
            if (field.isAnnotationPresent(NotBlank.class)) {
                if (value == null || value.trim().isEmpty()) {
                    errors.add(field.getName() + " cannot be blank");
                }
            }

            // Check MaxLength
            if (field.isAnnotationPresent(MaxLength.class)) {

                MaxLength max = field.getAnnotation(MaxLength.class);

                if (value != null && value.length() > max.value()) {
                    errors.add(field.getName()
                            + " is too long. Maximum "
                            + max.value() + " characters allowed");
                }
            }
        }

        if (errors.isEmpty()) {
            System.out.println("Form is valid");
        } else {
            for (String error : errors) {
                System.out.println(error);
            }
        }
    }
}
```

OUTPUT

```
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 7> javac Main.java
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 7> java Main
username cannot be blank
email is too long. Maximum 20 characters allowed
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 7>
```

| | |
|---|---|
| PROBLEM2 | 2. Mini test runner: define a @Run annotation and mark some no-arg methods of a class with it. Use reflection to find and invoke only the @Run methods and report how many ran — the idea behind how JUnit finds @Test methods. |
| CODE | |

```
import java.lang.annotation.*;
import java.lang.reflect.Method;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Run {
}

class MyTests {

   @Run
   public void test1() {
      System.out.println("Test 1 executed");
   }

   @Run
   public void test2() {
      System.out.println("Test 2 executed");
   }

   public void normalMethod() {
      System.out.println("Normal method");
   }
}

public class Mini {

   public static void main(String[] args) throws Exception {

      MyTests obj = new MyTests();

      int count = 0;

      Method[] methods = obj.getClass().getDeclaredMethods();

      for (Method method : methods) {

         if (method.isAnnotationPresent(Run.class)
               && method.getParameterCount() == 0) {

            method.invoke(obj);
            count++;
         }
      }
```

```
        System.out.println("Total methods executed: " + count);
    }
}
```

OUTPUT

```
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 7> javac Mini.java
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 7> java Mini
Test 2 executed
Test 1 executed
Total methods executed: 2
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 7>
```

PROBLEM3

3. (For Advanced Learners – home) Define a @Column(name) annotation and, by reflection, populate an object's fields from a header row and a data row by matching column names; decide how to handle a missing column.

CODE

```
import java.lang.annotation.*;
import java.lang.reflect.Field;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Column {
    String name();
}

class Student {

    @Column(name = "id")
    String id;

    @Column(name = "name")
    String name;

    @Column(name = "email")
    String email;
}

public class Columns {

    public static void main(String[] args) throws Exception {

        String[] headers = {"id", "name", "email"};
        String[] data = {"101", "Dharmik", "dharmik@gmail.com" };
```

```
        Student student = new Student();

        Field[] fields = student.getClass().getDeclaredFields();

        for (Field field : fields) {

            if (field.isAnnotationPresent(Column.class)) {

                Column column = field.getAnnotation(Column.class);

                for (int i = 0; i < headers.length; i++) {

                    if (headers[i].equals(column.name())) {

                        field.setAccessible(true);
                        field.set(student, data[i]);

                        break;
                    }
                }
            }
        }

        System.out.println("ID: " + student.id);
        System.out.println("Name: " + student.name);
        System.out.println("Email: " + student.email);
    }
}
```

OUTPUT

```
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 7> javac Columns.java
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 7> java Columns
ID: 101
Name: Om
Email: om@gmail.com
```